

Convex function interpolation.py

James Evans

07/22/2025

Note: `compute_beta_j_primal` and `compute_beta_star_primal` are alternative but mathematically equivalent implementations of Tony's `linear_programming.py` program located in the Interpolation Problem subfolder.

```
35
36 def compute_beta_j_dual(points, values1, values2, j):
37     N, d = points.shape
38
39     delta_x = points - points[j]          # shape (N, d)
40     delta_y = values1 - values1[j]        # shape (N,)
41     delta_yprime = values2 - values2[j]   # shape (N,)
42
43     lam = cp.Variable(N, nonneg=True)
44
45     # Constraints:
46     constraints = []
47
48     # sum_i lambda_i * (x_i - x_j) = 0
49     for k in range(d):
50         constraints.append(cp.sum(cp.multiply(lam, delta_x[:, k])) == 0)
51
52     # sum_i lambda_i * (y'_i - y'_j) = 1
53     constraints.append(cp.sum(cp.multiply(lam, delta_yprime)) == 1)
54
55     # Objective: minimize sum_i lambda_i * (y_i - y_j)
56     objective = cp.Minimize(cp.sum(cp.multiply(lam, delta_y)))
57
58     prob = cp.Problem(objective, constraints)
59     prob.solve()
60
61     return prob.value
62
```

def compute_beta_j_dual()

Given a collection of points $x_1, \dots, x_N \in \mathbb{R}^d$, associated scalar function values $y_i = f(x_i)$, and *auxiliary function values* $y'_i = g(x_i)$ (where g is a second scalar function evaluated at the same samples), the goal is to assess whether these samples are consistent with a convex function. Specifically, for a fixed reference point

x_j , we seek to construct a dual certificate that verifies whether the observed triplet (x_j, y_j, y'_j) is compatible with the existence of a convex interpolant of f (with y'_j used only for normalization).

Mathematical Setup

Define the differences:

$$\begin{aligned}\delta x_i &:= x_i - x_j \in \mathbb{R}^d, \\ \delta y_i &:= y_i - y_j \in \mathbb{R}, \\ \delta y'_i &:= y'_i - y'_j \in \mathbb{R}.\end{aligned}$$

Let $\lambda \in \mathbb{R}_{\geq 0}^N$ be a nonnegative weight vector. The optimization problem considered is:

$$\begin{aligned}\beta_j^* &:= \min_{\lambda \in \mathbb{R}_{\geq 0}^N} \sum_{i=1}^N \lambda_i \delta y_i \\ \text{subject to } &\sum_{i=1}^N \lambda_i \delta x_i = 0 \quad (\text{centering constraint}), \\ &\sum_{i=1}^N \lambda_i \delta y'_i = 1 \quad (\text{normalization via auxiliary values}).\end{aligned}$$

Centering Constraint

The constraint

$$\sum_{i=1}^N \lambda_i (x_i - x_j) = 0$$

says that the weighted sum of the vectors from x_j to each x_i must cancel out. In other words, the optimization is only allowed to combine the x_i in such a way that their net effect, relative to x_j , is zero.

Why is this important? Because x_j is the reference point: it's the location where we are testing whether the function value y_j can be explained by nearby data under convexity. If this constraint were not present, the optimization could shift attention to a different location — not x_j — and give a result that has nothing to do with what's happening at x_j .

This constraint prevents that. It ensures that the λ_i -weighted combination of points does not move the center of analysis away from x_j . That way, the test remains anchored at x_j , and we are truly checking whether the function is behaving convexly around that specific point.

Normalization via y'

The constraint

$$\sum_{i=1}^N \lambda_i (y'_i - y'_j) = 1$$

uses the auxiliary values $y'_i = g(x_i)$ purely to *fix the scale* of the weights. Each term $\delta y'_i = y'_i - y'_j$ is a scalar difference of the auxiliary function at x_i relative to x_j . The weights λ_i determine how these differences combine.

By requiring the total weighted auxiliary difference to equal 1, the optimizer cannot choose the trivial $\lambda = 0$ vector. Any other nonzero normalization (e.g. $\sum_i \lambda_i = 1$) would serve a similar scaling role; here we normalize with respect to y' as written in the code.

Objective

The objective

$$\sum_{i=1}^N \lambda_i (y_i - y_j)$$

computes a weighted sum of differences between each function value y_i and the reference value y_j . It quantifies how much higher the surrounding values are compared to y_j , according to the weights λ_i . The minimum of this expression gives the smallest such weighted discrepancy consistent with the constraints. A negative optimum indicates inconsistency with convexity at x_j .

```
62
63 def compute_beta_star_dual(points, values1, values2):
64     N = points.shape[0]
65     beta_array = np.zeros(N)
66
67     for j in range(N):
68         beta_array[j] = compute_beta_j_dual(points, values1, values2, j)
69
70     beta_star = np.min(beta_array)
71     return beta_star
72
```

def compute_beta_star_dual()

What the Code Computes

This function evaluates the dual convexity test at every sample point x_j , using the surrounding function values y_i (with y'_i only for normalization as above).

For each $j = 1, \dots, N$, it computes a quantity β_j^* by solving a linear program centered at x_j . This program checks whether the data near x_j is locally consistent with convexity.

Once all β_j^* values are computed, the function returns

$$\beta^* = \min_j \beta_j^*.$$

This value β^* represents the smallest (most critical) result across all test points. The code is explicitly looping through every point in the dataset, running the dual test at each one individually, collecting the results, and returning the worst-case value.

Why Take the Minimum over β_j^* ?

Each value β_j^* is defined as the optimal value of a minimization problem centered at a single point x_j :

$$\beta_j^* := \min_{\lambda \in \mathbb{R}_{\geq 0}^N} \sum_{i=1}^N \lambda_i (y_i - y_j) \quad \text{subject to constraints.}$$

This quantity measures how well the surrounding data supports convexity at x_j . If $\beta_j^* < 0$, it indicates a violation of convexity at that point. If $\beta_j^* \geq 0$, the local data is consistent with a convex function.

To evaluate whether the dataset as a whole is convex-consistent, we define

$$\beta^* := \min_j \beta_j^*.$$

This takes the worst-case value over all sample points. If $\beta^* < 0$, then convexity fails at at least one point. If $\beta^* \geq 0$, then all local tests pass.

Taking the minimum over j is therefore necessary: even a single failure invalidates convexity. The final value β^* acts as a global certificate—convexity is confirmed if and only if $\beta^* \geq 0$.